



CCA·F

CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

16-Week Study & Portfolio Plan

Beginner · Python track · 2 hrs/day × 7 days × 16 weeks = 224 hours

PROGRAM	Anthropic Claude Certified Architect — Foundations
FORMAT	60 questions · 120 minutes · scaled 100–1000 · pass 720
FEE	USD 99 (free for Anthropic partners)
PREPARED	May 2026
VERSION	1.0 · Style C

PRIVATE STUDY DOCUMENT

TOC

Contents

01 The Exam at a Glance**02 The Daily 2-Hour Block****03 The 16-Week Schedule**

Phase 1 • Foundations (Weeks 1–4) • Phase 2 • Project 1 (Weeks 5–6) • Phase 3 • Agents (Weeks 7–9) • Phase 4 • Project 2 (Weeks 10–11) • Phase 5 • Project 3 (Week 12) • Phase 6 • Claude Code + Project 4 (Weeks 13–14) • Phase 7 • Exam Prep (Weeks 15–16)

04 The Four Portfolio Projects

Project 1 • Structured Data Extractor • Project 2 • Customer Support Agent • Project 3 • Multi-Agent Research Pipeline • Project 4 • CI/CD Review Bot

05 Reading & Resource Checklist**06 Tooling Checklist****07 Exam-Day Tactical Notes****08 Weekly Self-Check Questions****09 Portfolio Polish Checklist****10 Risk Register**

Hyperlinks throughout this document are functional. Click any URL or section reference to navigate. External resources link to canonical Anthropic and open-source documentation.

01 The Exam at a Glance

The CCA-F is Anthropic's first architecture-level technical certification — a 60-question, 120-minute, scenario-based exam scored on a 100–1000 scale with a passing mark of 720. Anthropic's stated bar is six months of hands-on experience with the Claude API, Agent SDK, Claude Code, and Model Context Protocol (MCP). This plan compresses that into sixteen focused weeks by replacing passive reading with builds.

Register through the official access request page at <https://anthropic.skilljar.com/claude-certified-architect-foundations-access-request>. The fee is USD 99 for non-partners and free for organizations enrolled in the Claude Partner Network. Pay in Week 1 — booking the exam changes how you study, and you gain immediate access to Anthropic's official practice questions on the Skilljar platform.

EXHIBIT 1 • DOMAIN WEIGHTING (5 DOMAINS, 100% TOTAL)

CODE	DOMAIN	WEIGHT
D1	Agentic Architecture & Orchestration	27%
D2	Tool Design & MCP Integration	18%
D3	Claude Code Configuration & Workflows	20%
D4	Prompt Engineering & Structured Output	20%
D5	Context Management & Reliability	15%

EXHIBIT 2 • SIX PRODUCTION SCENARIOS (4 RANDOMLY DRAWN PER SITTING)

#	SCENARIO	DESCRIPTION	PRIMARY
S1	Customer Support Resolution Agent	Agent SDK + custom MCP tools, 80%+ first-contact resolution, escalation logic.	D1, D2, D5
S2	Code Generation with Claude Code	CLAUDE.md, plan mode, iterative refinement.	D3, D1, D5
S3	Multi-Agent Research Pipeline	Hub-and-spoke orchestration, subagent delegation.	D1, D5
S4	Developer Productivity Tools	Codebase exploration agent with MCP integration.	D2, D3, D1
S5	Claude Code in CI/CD	Headless review bot, structured output, low false positives.	D3, D4

#	SCENARIO	DESCRIPTION	PRIMARY
S6	Structured Data Extraction	JSON schema validation, retry loops on unstructured documents.	D4, D5

KEY FINDING

Every scenario question rewards muscle memory you only get from shipping code. The candidates who pass aren't the ones who watched the most videos — they're the ones who, on exam day, recognize each scenario because they've already built the system. By the end of Week 14 you will have built four of them.

02 The Daily 2-Hour Block

Use the same structure every day. Discipline on the block is the single highest-leverage habit in this plan; without it, the 224 hours dissolve into background reading and you arrive at the exam unprepared. With it, you arrive with four shipped projects and muscle memory across every domain.

EXHIBIT 3 • THE 120-MINUTE DAILY BLOCK

MINUTES	ACTIVITY	OUTPUT
00 – 10	Warm-up: re-read yesterday's notes, queue today's goals.	Day plan
10 – 40	Theory: course video, official documentation, scenario walk-through.	Notes draft
40 – 110	Lab: hands-on coding. Type, do not copy. No shortcuts.	Working code
110 – 120	Capture: structured notes in <code>notes/<domain>.md</code> , commit to your prep repo.	Git commit

REPO STRUCTURE

Set up a single GitHub repository `cca-prep/` on Day 1. Folders: `notes/` , `labs/` , `project-01-extractor/` , `project-02-support-agent/` , `project-03-research/` , `project-04-ci-bot/` , `mocks/` . No commit on a study day = the day did not count.

03**The 16-Week Schedule**

The schedule runs in seven phases. Phases 1, 3, 6 build skill; Phases 2, 4, 5 deliver portfolio projects; Phase 7 converts skill plus portfolio into an exam pass. Total: 224 hours across 112 study days. No skipped days — see Section 10 (Risk Register) for the discipline rationale.

Phase 1 Foundations

Weeks 1-4 • 56 hours

Week 01 Claude API & Development Environment

D4

D5

Mon-Tue	Install Python 3.11+, anthropic-sdk-python . Read Anthropic Academy Building with the Claude API Modules 1-2. Build first <code>client.messages.create()</code> calls in a Jupyter notebook.
Wed	Learn message structure: <code>role</code> , <code>content</code> blocks (text, image, tool_use, tool_result, document). Build a 3-turn CLI chat loop.
Thu	Streaming with <code>client.messages.stream()</code> . Compare latency, streaming vs non-streaming.
Fri	System prompts, <code>temperature</code> , <code>max_tokens</code> , <code>stop_sequences</code> . A/B a classification prompt at temp 0 vs 1.
Sat	Token counting with <code>client.messages.count_tokens()</code> . Build a cost-estimator script for batch jobs.
Sun	Write <code>notes/01-api-foundations.md</code> . Push to GitHub.

Milestone Working CLI chatbot, streaming demo, and token-counter script committed to `labs/week-01/`.

Week 02 Prompt Engineering & Structured Output

D4

Mon	Read Anthropic prompt engineering docs end-to-end.
Tue	Few-shot prompting. Build a sentiment classifier; measure 0-shot vs 5-shot accuracy on 50 hand-labeled examples.
Wed	Structured output the right way: enforce JSON via system prompt plus assistant prefill <code>{</code> — there is no <code>response_format</code> parameter on the Messages API. Memorize this pattern.
Tue	Introduce Pydantic v2 . Define schemas, validate every response, log failures.
Fri	Validation retry loop — on schema failure, re-prompt with the error. Cap at 3 retries. Exam-critical pattern.
Sat	Prompt caching with <code>cache_control</code> . Cache a 50-page PDF, ask 10 questions; measure savings.
Sun	Notes plus official prompt library review.

Milestone `labs/week-02/structured_classifier.py` with Pydantic validation and bounded retry loop.

Week 03 Tool Use & Function Calling

D2

Mon	Read tool-use docs. Understand <code>tools</code> parameter, JSON-Schema definitions, <code>tool_choice</code> (<code>auto</code> / <code>any</code> / <code>tool</code>).
Tue	Single-tool agent: <code>get_weather(location)</code> . Implement the full loop: <code>tool_use</code> → <code>execute</code> → <code>tool_result</code> → <code>response</code> .
Wed	Multi-tool agent: add <code>get_news</code> , <code>convert_currency</code> . Observe tool selection.
Thu	Tool design principles: descriptive names, narrow scope, exhaustive descriptions, examples-in-description. Rewrite weather tool to exam quality.
Fri	Parallel tool use. Handle multiple concurrent results.
Sat	Error handling. Return <code>is_error: true</code> <code>tool_result</code> blocks; observe recovery.
Sun	Notes. Key concept: tool descriptions are prompts — invest in them.

Milestone `labs/week-03/multi_tool_agent.py` with ≥4 tools, parallel use, error recovery.

Week 04 Model Context Protocol (MCP)

D2

Mon	Read MCP specification . Memorize primitives: tools , resources , prompts , sampling , roots .
Tue	Install mcp Python SDK . Run the official everything server via npx. Connect Claude Desktop.
Wed	Write first MCP server: a 'personal notes' server with one tool (<code>add_note</code>) and one resource (<code>notes://list</code>).
Thu	Add prompts. Transports: stdio (local, simple) vs SSE / streamable HTTP (remote, networked). Know the trade-offs.
Fri	Authentication patterns: OAuth 2.1 flow for remote servers, bearer tokens, scopes.
Sat	Build an MCP <i>client</i> in Python. Connect to your own server programmatically.
Sun	Write an MCP cheat sheet you could explain on a whiteboard.

Milestone [labs/week-04/notes-mcp-server/](#) — full server plus client, runs via stdio, documented README.

Phase 2 First Build

Weeks 5–6 • 28 hours

Weeks 5–6 • 28 hours

P1

Portfolio Project 1 • Structured Data Extractor

Covers Scenario 6 • Domains D4 (heavy), D5

Week 5: Architecture plus happy path. Build the schema, prompt, and basic extraction over 20 sample invoices.

Week 6: Production hardening — retry loops, confidence scoring, batch mode, eval suite, CLI, README. Ship to GitHub.

Full build specification: see [Section 04 • Project 1](#).

Phase 3 Agents & Orchestration

Weeks 7–9 • 42 hours

Week 07 The Agentic Loop & Claude Agent SDK

D1

Mon	Read Building Effective Agents (Anthropic). Memorize: prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer.
Tue	Install claude-agent-sdk . Run quickstart. Understand the tool-loop abstraction.
Wed	Hooks: <code>PreToolUse</code> , <code>PostToolUse</code> , <code>UserPromptSubmit</code> , <code>Stop</code> . Build a JSONL logging hook.
Thu	Lifecycle: when does the agent stop? Read SDK source. Implement a custom stop condition.
Fri	Permission modes (<code>default</code> , <code>acceptEdits</code> , <code>plan</code> , <code>bypassPermissions</code>) — critical for the support-agent scenario.
Sat	Write a system prompt for a refund-handling agent: role, tools, decision policies, escalation criteria.
Sun	Notes plus a hand-drawn diagram of the agent loop.

Milestone `labs/week-07/sdk_agent_with_hooks.py` .

Week 08 Subagents & Orchestration Patterns

D1

Mon	Read How we built our multi-agent research system . Memorize hub-and-spoke.
Tue	Subagent fundamentals: separate context windows, focused prompts, structured handoff.
Wed	Build a 2-agent system: a researcher subagent called by a writer lead agent.
Thu	Task decomposition: by subject, by step, by viewpoint. Know when each fits.
Fri	When <i>not</i> to use subagents: simple tasks, latency-critical paths, conversational flows. Recognize the anti-pattern.
Sat	Read about Agent Skills (<code>SKILL.md</code> with YAML frontmatter, progressive disclosure). Build a small skill.
Sun	Draw hub-and-spoke vs orchestrator-worker diagrams from memory.

Milestone `labs/week-08/orchestrator_workers/` runs end-to-end.

Week 09 Context Management & Reliability

D5

Mon	Context window mechanics: 200k tokens but quality degrades on the long tail. Strategies: summarization, scratchpad, subagent delegation, retrieval.
Tue	Scratchpad pattern — agent writes intermediate state to disk, frees context. Implement it.
Wed	Compaction — periodic summarization. Trigger at 70% context full.
Thu	Memory hierarchies: short-term (turn), working (task), long-term (vector store or files). Add a memory layer to one project.
Fri	Escalation patterns: confidence thresholds, policy gaps, capability limits, customer-requested. Design a rubric.
Sat	Reliability primitives: timeouts, exponential backoff, circuit breakers, idempotency keys. Apply to Week-7 agent.
Sun	Notes. Exam tip: context-overflow questions are answered by extracting structured state, not by 'increase max_tokens'.

Milestone Week-7 agent upgraded with scratchpad, compaction, and structured error recovery.

Phase 4 Agent Build

Weeks 10–11 • 28 hours

Weeks 10–11 • 28 hours

P2**Portfolio Project 2 • Customer Support Agent + MCP Server**

Covers Scenario 1 • Domains D1, D2, D5

Week 10: Build a FastAPI mock backend with fake customers, orders, refund logic. Build a custom MCP server exposing it. Wire up a basic agent.

Week 11: Add escalation logic, conversation memory, eval harness with 30 hand-written test cases. Document and ship.

Full build specification: see [Section 04 • Project 2](#).

Phase 5 Multi-Agent

Weeks 12 • 14 hours

Week 12 • 14 hours

P3**Portfolio Project 3 • Multi-Agent Research Pipeline**

Covers Scenario 3 • Domain D1 (deep)

Seven days, sprint-style. By Sunday the pipeline answers a real research question end-to-end: lead agent decomposes the question, subagents run in parallel, lead aggregates structured findings into a final report.

Full build specification: see [Section 04 • Project 3](#).

Phase 6 Claude Code + Project 4

Weeks 13–14 • 28 hours

Week 13 Claude Code Configuration & Workflows

D3

- Mon** Install Claude Code (`npm install -g @anthropic-ai/claude-code`). Run on an existing project. Learn `/help` , `/clear` , `/compact` , `/cost` .
- Tue** **CLAUDE.md hierarchy**: `~/.claude/CLAUDE.md` (user) → `./CLAUDE.md` (project root) → `./<subdir>/CLAUDE.md` . Memorize precedence — exam favorite.
- Wed** Write CLAUDE.md for a polyglot repo using *scoped CLAUDE.md per subdirectory*, not one root file.
- Thu** **Plan mode** vs direct execution. Plan mode for large refactors, exploration, architectural changes. Practice on a refactor.
- Fri** Custom slash commands in `.claude/commands/` — Markdown with optional YAML frontmatter. Write `/review` , `/test` , `/explain` .
- Sat** MCP servers inside Claude Code: `.mcp.json` , `claude mcp add` , `scopes ( local , project , user )` . Connect a server.
- Sun** Claude Code hooks: `PreToolUse` , `PostToolUse` , `Stop` , `SessionStart` . Write one that auto-formats with ruff.

Milestone `cca-prep` repo fully configured: CLAUDE.md, 3 slash commands, 1 hook, 1 MCP server.

Week 14 • 14 hours

P4 Portfolio Project 4 • CI/CD Review Bot
Covers Scenario 5 • Domains D3, D4

- Mon–Wed**: Build headless Claude Code in a GitHub Action, triggered on PR. Output structured JSON review comments.
- Thu–Fri**: Prompt-engineer for minimal false positives: explicit review criteria, severity levels, multi-instance review pattern (generator plus independent reviewer).
- Sat–Sun**: Test on 10 real PRs, measure precision and recall, document and ship.

Full build specification: see Section 04 • Project 4.

Phase 7 Exam Preparation

Weeks 15-16 • 28 hours

Week 15 Domain Review & First Mock

All

Mon	Take a full timed 60-question mock (official practice plus one third-party set). Grade ruthlessly. Mark weak domains.
Tue	Re-read the official Exam Guide PDF end-to-end. Highlight every task statement under each domain.
Wed-Fri	Deep-dive your two weakest domains. Re-do the relevant Phase 1-6 lab. If D1 is weak, rebuild a small orchestrator from scratch.
Sat	Scenario drills — write a 1-page architecture sketch for each of the 6 scenarios from memory. Compare against your project work.
Sun	Second mock, different question bank. Aim for 80%+.

Milestone Confirmed weakest domain identified and remediated; two mock results logged.

Week 16 Final Polish & Exam Day

All

Mon	Mock #3, full timed. Aim 85%+.
Tue	Targeted drilling on every question missed across all three mocks.
Wed	Build your Master Cheat Sheet : one printable page covering MCP primitives, Agent SDK hooks, CLAUDE.md hierarchy, JSON-schema retry pattern, escalation rubric, scratchpad pattern, plan-mode triggers, subagent decomposition. Read nightly.
Thu	Light review only. Sleep.
Fri	Light review. Verify exam environment: clean room, photo ID, working webcam, stable internet, ≥30 min lead time. Sleep.
Sat	Exam day. Eat. Hydrate. Read each question twice. Eliminate two distractors. Flag-and-skip anything over 2 minutes. Never leave a question blank.
Sun	Decompress. Push polished READMEs to all 4 portfolio repos. Add the credential badge to LinkedIn within 48 hours.

Milestone Pass the exam. Portfolio repos polished and credential publicly visible.

04 The Four Portfolio Projects

Each project lives in its own repository. Across the four, every CCA-F domain is exercised and four of the six exam scenarios are directly built. The remaining two scenarios (Code Generation with Claude Code, Developer Productivity Tools) are covered by your Week 13 Claude Code practice and your daily lab work.

EXHIBIT 4 • STANDARD PROJECT REPOSITORY STRUCTURE

```
project-XX-name/  
├─ README.md           # problem, architecture diagram, demo gif, results  
├─ pyproject.toml      # uv or poetry  
├─ src/<package>/      # source  
├─ tests/              # pytest suite  
├─ evals/              # eval harness + dataset  
├─ examples/           # sample inputs/outputs  
└─ docs/               # ADRs (architecture decision records)
```

Every README opens with these sections in this order: **Problem, Architecture diagram, Quickstart (3 commands), Eval results table, Trade-offs and known limits, What I would do next**. Recruiters skim — that order is deliberate.

PROJECT 01

Structured Data Extractor with Validation Retry Loop

Scenario: Extract typed data from messy unstructured documents (invoices, contracts, lab reports) with measurable accuracy.

Maps to: Scenario 6 • D4 (heavy), D5

Stack: Python 3.11 • `anthropic` • `pydantic` • `pytest` • `typer` • `rich`

BUILD STEPS

- 01 **Pick a target document type.** I recommend US-style invoices (50+ samples readily available on Kaggle). In your README, state the business value crisply: 'Accountants spend X hours/week typing invoices into QuickBooks; this extracts to JSON in 5 seconds with 95%+ field accuracy.'
- 02 **Design the schema first, in Pydantic.** Fields: vendor name, vendor address, invoice number, invoice date, line items (list), subtotal, tax, total, currency. Use `Annotated` types with `Field(description=...)` — Claude reads the docstrings via the schema.
- 03 **Build a hand-labeled gold set.** 50 documents with ground truth in `evals/gold.jsonl`. This is the most important step. Do not skip. Without ground truth you cannot measure improvement.
- 04 **Write the extractor v0.** Single function `extract(doc_path) -> Invoice`. PDF → text via `pypdf`, or image-mode via the Claude vision API (preferred for messy scans). Inline the Pydantic schema as JSON Schema in the system prompt.
- 05 **Prefill the assistant response with `{` to force JSON.** Set `stop_sequences=["``"]` if you wrap in fences. Parse `json.loads`, then `Invoice.model_validate(...)`.
- 06 **Add the validation retry loop.** On `ValidationError`, re-prompt with the error message and the original document, asking Claude to fix only the bad fields. Cap at 3 retries. Log every retry. This is the canonical exam pattern.
- 07 **Add confidence scoring.** Ask Claude to return a `confidence` field per row (`high / medium / low`) with a rationale. Route low-confidence outputs to a `for_human_review/` folder.
- 08 **Build an eval harness.** `python -m evals` runs the extractor on the gold set, computes per-field accuracy, dumps a Markdown report. Track results across prompt revisions in `evals/history.md`.
- 09 **Add prompt caching.** Your system prompt with the JSON Schema plus 3 few-shot examples is large and static — wrap in `cache_control`. Measure cost reduction in the eval report.

-
- 10 **Build a typer CLI.** `extract single file.pdf`, `extract batch ./invoices/`, `extract eval`. Pretty-print with rich.
 - 11 **Write tests.** Schema tests, retry-loop tests with mocked failures, golden-output regression tests.
 - 12 **Production hardening.** Timeouts, exponential backoff on 429/529, structured logging via structlog, max-spend guardrail per batch.
 - 13 **README plus demo.** GIF of the CLI in action, eval results table, architecture diagram (PlantUML or Excalidraw).
 - 14 **Stretch goal (pick one):** swap PDF → image-mode and measure quality delta · add an LLM-as-judge eval for the description field · add a small Streamlit web UI.
-

DONE WHEN

≥95% per-field accuracy on the gold set, retry loop demonstrably recovers from injected schema violations, repository has green CI.

PROJECT 02

Customer Support Resolution Agent with Custom MCP Server

Scenario: An agent that handles refunds, order lookups, and account questions, with safe tool use and clear escalation paths.

Maps to: Scenario 1 • D1, D2, D5

Stack: Python • `claude-agent-sdk` • `mcp` • `fastapi` • `sqlmodel` • `pytest`

BUILD STEPS

- 01 Build the mock backend first.** FastAPI app with endpoints `GET /customers/{id}`, `GET /orders/{id}`, `POST /refunds`, `GET /policies`. SQLite via `SQLModel`, seeded with 100 customers, 500 orders. Include tricky cases: cancelled orders, partial refunds already issued, blocked accounts, items past the return window.
- 02 Build the MCP server.** Five tools exposing the backend: `get_customer`, `lookup_order`, `get_order_history`, `process_refund`, `escalate_to_human`. Run over stdio. Use the [MCP Python SDK](#). **Tool descriptions are critical** — for `process_refund`, specify: must verify order belongs to customer, must check return window, must check no existing refund, max \$500 without escalation.
- 03 Read your own tool schemas back.** If you cannot tell from the schema alone whether to use a tool, neither can Claude. Iterate descriptions until they pass that test.
- 04 Build the agent with the [Claude Agent SDK](#).** System prompt defines: role (support agent for 'Acme Corp'), tone, escalation triggers, policy summary, available tools. Connect the MCP server.
- 05 Codify the escalation rubric in the system prompt.** Escalate when (a) policy is ambiguous, (b) refund > \$500, (c) customer explicitly asks for a human, (d) account is blocked, (e) confidence is low after 2 tool calls. Vague escalation rules are the #1 cause of bad support agents — and a common exam trap.
- 06 Wire up hooks.** `PreToolUse` logs every tool call. `PostToolUse` redacts PII (emails, card numbers). A `Stop` hook fires `escalate_to_human` if the agent loops on the same tool 3+ times.
- 07 Add a scratchpad.** Long conversations overflow context. After every 10 turns, summarize resolved facts to a scratchpad file the agent reads at turn start. Drop the raw history from context.
- 08 Build the eval harness.** 30 hand-written scenarios as JSON: `{situation, customer_message, expected_outcome, expected_tool_calls}`. Run each, score outcome match.
- 09 Measure first-contact resolution.** Target 80%+. If lower, the fixes are almost always tighter tool descriptions, clearer escalation triggers, better system-prompt examples — not a bigger model.

-
- 10 **Add safety rails.** `process_refund` returns `is_error: true` if amount > \$500 and the agent did not first call `escalate_to_human`. Test recovery.
-
- 11 **Build a chat CLI** for demo. Bonus: a tiny Gradio web UI.
-
- 12 **Document everything.** Architecture diagram showing Agent → SDK → MCP Server → FastAPI → SQLite. Eval table. Decision log explaining each escalation rule.
-
- 13 **Stretch goals:** add a 'human review queue' web page • add a confidence-routed multi-instance review (a second Claude inspects the agent's plan before tool execution) • stream the agent's intermediate steps to the user.
-

DONE WHEN

80%+ first-contact resolution on the eval set, zero refunds issued without verification, escalation triggers documented and tested.

PROJECT 03

Multi-Agent Research
Pipeline

Scenario: A lead agent decomposes a research question, delegates to specialist subagents in parallel, aggregates results.

Maps to: Scenario 3 • D1 (deep) • D5

Stack: Python • `claude-agent-sdk` • web-search tool (Anthropic native or Brave Search) • `pydantic`

BUILD STEPS

- 01 **Pick a research domain.** Examples: 'competitive landscape for X SaaS product', 'literature review for Y topic', 'due-diligence summary on Z company'. Pick one and commit — depth beats breadth.
- 02 **Design the agent topology.** Hub-and-spoke: one **Lead** agent (planner plus aggregator) and N **Researcher** subagents (each owns a sub-question). Diagram it before coding.
- 03 **Implement task decomposition.** Lead receives the user question and returns a structured `ResearchPlan` (Pydantic): list of sub-questions, each with rationale, search strategy, deliverable shape. Force JSON output via prefill.
- 04 **Spin up N parallel subagents**, one per sub-question. Each gets a system prompt scoped to *just* its sub-question, its own context window (this is the point), and a web-search tool. Use `asyncio.gather` for parallelism.
- 05 **Define a structured handoff schema.** Each subagent returns `SubagentReport : findings , sources , confidence , gaps`. Lead aggregates only these typed objects — never raw subagent transcripts. **This is the heart of hub-and-spoke** and shows up in scenario questions.
- 06 **Lead synthesizes.** Receives all reports, dedupes findings, resolves contradictions, writes the final report. If two subagents disagree, lead dispatches a tie-breaker subagent or flags the disagreement explicitly.
- 07 **Handle subagent failure.** If a subagent returns low confidence or hits a tool error, lead retries with a refined sub-question once, then proceeds with the rest and notes the gap in the final report.
- 08 **Add a scratchpad for the lead.** Plan plus each subagent report goes to `runs/<run_id>/`. Lead reads structured summaries from disk, not full subagent histories.
- 09 **Token budget per subagent.** Cap each at e.g. 50k tokens of context. On overflow, force a self-summary and continue. Demonstrate awareness of context economics — it is tested.
- 10 **Eval set.** 10 research questions with rough expected coverage (key entities to mention, key sources to find). Score: coverage %, citation count, contradictions handled.

-
- 11 **Add a viewer.** A small CLI or web page that shows the full run — plan, each subagent's findings, final report — for transparency. Recruiters love this.
 - 12 **Document:** architecture diagram, sample run (PDF or HTML), cost-per-run breakdown.
 - 13 **Stretch goals:** add an **evaluator-optimizer** loop — a critic subagent reviews the final report and the lead revises once · swap one subagent for a SQL-querying specialist over a real dataset.
-

DONE WHEN

Runs end-to-end on at least 5 questions, produces a clean Markdown report with citations, total run cost under \$0.50 per question.

PROJECT 04

Claude Code CI/CD Review Bot

Scenario: Headless Claude Code reviews every pull request, leaves structured comments, minimizes false positives.

Maps to: Scenario 5 • D3, D4

Stack: Claude Code (headless) • GitHub Actions • Python • GitHub REST API

BUILD STEPS

- 01 **Pick a victim repo.** Fork a moderately active open-source Python repo (or use your own). You need real PRs to test on. Set up GitHub Actions.
- 02 **Get Claude Code running headless.** Locally first: `claude --print "Review this diff: ..." --output-format json --allowed-tools "Read,Grep,Glob"`. Get familiar with `--print`, `--output-format`, `--allowed-tools`, `--add-dir`.
- 03 **Write CLAUDE.md for the target repo.** Project conventions, language versions, testing approach, a 'never suggest' list. Crucial for false-positive reduction.
- 04 **Write `.claude/commands/review.md`** as a custom slash command with explicit review criteria: correctness, security, perf, style, test coverage. Each finding has severity (`critical|major|minor|nit`) and rationale. Forbid nits unless severity $\geq$ minor count = 0.
- 05 **Design the structured output.** A `ReviewReport` Pydantic schema: `summary`, `findings: list[Finding]` with `(file, line, severity, category, message, suggestion)`. Force JSON via prefill.
- 06 **Build the GitHub Action.** Triggers on `pull_request`. Steps: checkout, install Claude Code, fetch PR diff, run `claude --print` with the review prompt, parse JSON, post one review with line-anchored comments via GitHub REST API. Use `ANTHROPIC_API_KEY` from repo secrets.
- 07 **Multi-instance review for low false-positive rate.** Run two independent invocations: Reviewer-A scans the diff; Reviewer-B receives A's findings plus the diff and keeps only what it independently confirms. This is the canonical pattern for self-review's blind spot.
- 08 **Suppress duplicates against existing tests.** Before suggesting a new test, the prompt asks Claude to first read the existing test file and check whether the scenario is already covered. Major false-positive killer.
- 09 **Severity-aware posting.** Only `critical` and `major` as blocking comments; `minor` as suggestions; `nit` as a single summary block. Configurable per repo via `.claude/review.config.json`.
- 10 **Cache the project context.** CLAUDE.md plus conventions are static — leverage prompt caching so each PR review only pays for the diff.

-
- 11 **Build an offline eval.** 10 real PRs you have manually reviewed. Score precision and recall. Tune prompts until precision $\geq 80\%$.
 - 12 **Add a `/explain` companion command** that engineers can invoke in any line comment for follow-ups. Keeps the bot useful, not just noisy.
 - 13 **Document with a real PR example.** Screenshot a real PR with bot comments, link to the run log. Show before/after false-positive rate.
-

DONE WHEN

Runs on real PRs without manual intervention, precision $\geq 80\%$ on the eval set, false-positive rate on critical and major findings $\leq 10\%$.

05

Reading & Resource Checklist

Consume each resource during the week it maps to. Do not binge upfront — passive reading without an attached lab is the most common failure mode (see Risk Register).

EXHIBIT 5 • OFFICIAL ANTHROPIC RESOURCES

WHEN	RESOURCE	LINK
Pre-W1	CCA-F Exam Guide PDF (the source of truth — read 3× during prep)	Skilljar exam page
W1-W3	Anthropic Academy • <i>Building with the Claude API</i> (8h flagship course)	anthropic.skilljar.com
W4	Anthropic Academy • <i>Introduction to MCP + MCP: Advanced Topics</i>	anthropic.skilljar.com
W4	Model Context Protocol specification	modelcontextprotocol.io
W7	Anthropic Engineering • <i>Building Effective Agents</i>	anthropic.com
W7	Anthropic Engineering • <i>Building agents with the Claude Agent SDK</i>	anthropic.com
W8	Anthropic Engineering • <i>How we built our multi-agent research system</i>	anthropic.com
W8	Anthropic Academy • <i>Introduction to Subagents + Introduction to Agent Skills</i>	anthropic.skilljar.com
W9	Anthropic Engineering • <i>Demystifying evals for AI agents</i>	anthropic.com
W13	Claude Code documentation (full reference)	docs.claude.com
W2 + W6	Anthropic prompt library	docs.claude.com
Ongoing	Anthropic engineering blog index	anthropic.com/engineering
Ongoing	Claude API documentation root	docs.claude.com

EXHIBIT 6 • PRACTICE QUESTION BANKS (FINAL 4 WEEKS ONLY)

- **Official:** Anthropic's practice exam (in your Skilljar account after registration). Use first.
- **Community:** Pick two well-reviewed third-party banks. Do not memorize answers — understand *why* each wrong answer is wrong. That is the testable skill.

EXHIBIT 7 • SUPPORTING REFERENCES

- [Pydantic v2 documentation](#) — used in every project.
- [anthropic-sdk-python \(GitHub\)](#) — official Python SDK source.
- [claude-agent-sdk-python \(GitHub\)](#) — read the loop source in Week 7.
- [MCP Python SDK \(GitHub\)](#) — used in Projects 2 and 4.
- *Designing Data-Intensive Applications* — reliability and retries chapters only, optional but high-yield.

06 Tooling Checklist

Set this up on Day 1 of Week 1 and do not touch it again. The point is to make every subsequent day frictionless — the worst failure mode in a 16-week plan is daily tooling friction that erodes the 2-hour block.

EXHIBIT 8 • DAY-1 TOOLING SETUP

CATEGORY	TOOL	PURPOSE
Language	Python 3.11+ via uv or pyenv	Standard runtime for all projects.
Packaging	uv (preferred) or poetry	Project and dependency management.
Quality	ruff, pyright, pytest, pytest-asyncio	Lint, types, tests across every project.
Hooks	pre-commit with ruff and pyright	Catch issues before commit.
Env	direnv	Per-project .env loading.
VCS	GitHub org or user with cca-prep + 4 project repos	One canonical home for the journey.
CLI	gh CLI, authenticated	PR creation, repo operations.
AI Coding	Claude Code with personal ~/.claude/CLAUDE.md	Document your preferences once.
Evals	Reusable pattern: evals/gold.jsonl + evals/run.py + Markdown report	Same eval scaffold across all 4 projects.

07

Exam-Day Tactical Notes

These ten tactics translate domain knowledge into the marginal points that separate 720 from 700. Memorize them. Many failed attempts come from candidates who knew the material but spent badly: too long on early questions, no distractor elimination, blanks left on the table.

EXHIBIT 9 • TEN TACTICAL RULES

01	Read each scenario stem twice before the questions.	Scenarios are dense; the right answer often hinges on a single constraint buried in the stem.
02	Eliminate two distractors before picking.	Most CCA distractors are plausible-looking but violate one principle (e.g. 'increase max_tokens to fix context overflow' — wrong category of fix).
03	Watch for over-engineering distractors.	Anthropic's correct answer is often the simpler architectural choice. A response that adds 3 subagents to solve a single-prompt problem is the trap.
04	Plan mode in Claude Code questions.	Choose plan mode whenever the task involves exploration, multi-file refactors, or architectural decisions; choose direct execution for narrow, well-specified edits.
05	Subagents — use sparingly.	Use when the task has independent sub-problems, requires fresh context, or benefits from parallelism. Do NOT use for chatty or sequential work where context continuity matters.
06	JSON enforcement questions.	The pattern is <i>prompt + schema in system prompt + assistant prefill + validation + bounded retry</i> . Any answer mentioning 'use a parameter to force JSON' is wrong — no such parameter exists on the Messages API.
07	Tool-description questions.	The right fix is almost always 'improve the tool description / add examples', not 'switch model' or 'lower temperature'.
08	Escalation questions.	The right answer routes by <i>confidence + policy + customer request</i> , not by a hard turn-count.
09	Pace.	120 minutes ÷ 60 questions = 2 minutes each. Aim for 90 seconds on the first pass; leave 30 minutes for flagged questions.
10	No blank answers.	

No guessing penalty. Every blank is wasted EV. Eliminate two distractors, pick the better remaining option, move on.

08

Weekly Self-Check Questions

At the end of each week, write a one-paragraph answer (in your own words) to that week's question. If you cannot answer cleanly, the week did not stick — repeat the key lab before moving on. These questions are intentionally written to mirror exam-question framing.

EXHIBIT 10 • WEEKLY KNOWLEDGE CHECKS

WHEN	QUESTION
Week 1	What goes in <code>messages</code> vs <code>system</code> vs <code>tools</code> ?
Week 2	Why is 'use <code>response_format: json</code> ' wrong on the Messages API, and what is the correct pattern?
Week 3	What makes a tool description 'production-grade'? Give five properties.
Week 4	Stdio vs streamable HTTP — when do you pick each? What auth model does each support?
Week 5–6	How does your retry loop avoid infinite recursion <i>and</i> hopeless re-prompting?
Week 7	Walk through one full iteration of the agent loop, naming every message-block type sent.
Week 8	When is hub-and-spoke wrong, and what replaces it?
Week 9	Three distinct fixes for context overflow, ranked by when to apply.
Week 10–11	What is your refund agent's escalation rubric, verbatim?
Week 12	How does a structured subagent report shape avoid context pollution in the lead?
Week 13	Draw the CLAUDE.md precedence chain on paper.
Week 14	Why does multi-instance review catch issues that self-review misses?

09

Portfolio Polish Checklist

Week 16, final four hours, per repository. Recruiters and hiring managers spend under 60 seconds on a portfolio README before deciding whether to open the code. Optimize for that first minute.

EXHIBIT 11 • PER-REPO POLISH CHECKLIST

- ☐ README opens with the **business problem** (one sentence), not the tech stack.
- ☐ Architecture diagram (Excalidraw export → PNG checked into the repo).
- ☐ Quickstart works in 3 commands. Test it on a fresh machine or Codespace.
- ☐ Eval results table with before/after numbers.
- ☐ A 30-second demo GIF or Loom link.
- ☐ An ADR (docs/adr/0001-...md) explaining one non-obvious trade-off.
- ☐ CI green (GitHub Actions running tests plus lint).
- ☐ License plus a clear 'this is a portfolio project, not maintained' note.
- ☐ LinkedIn post drafted referencing the four repos and the CCA-F badge.

PUBLIC PROOF

Within 48 hours of passing, add the credential to LinkedIn and publish a short post linking the four project repositories. Hiring conversations begin in the comments of that post more often than they begin in cold outreach.

10

Risk Register

Read once at Week 1. Re-read in Week 8 (mid-program). Each risk includes a specific mitigation; if a risk materializes without a known mitigation, the plan is failing, and you should stop and redesign rather than push through.

EXHIBIT 12 • RISKS AND MITIGATIONS

RISK	MITIGATION
You 'study' by reading without coding	Phase 1 forces a daily commit. No commit on a study day = the day did not count.
Project 1 takes 3 weeks instead of 2	Set a Week-6-Sunday hard stop. Ship imperfect; move on.
Anthropic Academy course list changes	The exam guide PDF is canonical; courses are scaffolding. Trust the PDF.
You memorize practice-bank answers	Always ask 'why are the wrong answers wrong'. That is the testable skill.
Burnout around Week 9	Take Week-6 Sunday and Week-12 Sunday fully off. Non-negotiable.
Exam updates (new domains, weighting changes)	Re-verify the exam guide PDF in Week 14. Anthropic versions it.
Tooling drift mid-project	Pin versions in <code>pyproject.toml</code> . Snapshot working envs before risky upgrades.
Lab quality drift	Every Sunday, read your own week's notes back. If you cannot reconstruct the lab from notes alone, you do not own the concept yet.

CLOSING

The candidates who pass are not the ones who watched the most videos. They are the ones who, on exam day, recognize each scenario because they have already built the system. By the end of Week 14 you will have built four of them. The exam will feel like reviewing your own commit history. Good luck — and ship the projects.